

Lab 00 · Hola mundo

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

15 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que una aplicación Spring Boot arranque, y que lo primero que imprime lo escribiste tú. Maven y el JDK viajan dentro del repositorio del curso. El único requisito es Git.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. La oficina, que es la metáfora de este laboratorio	3
3. Los pasos	4
3.1. Paso 1 · Correr algo que todavía no es tuyo	4
3.2. Paso 2 · Que imprima algo tuyo	5
3.3. Paso 3 · Cambiar algo sin tocar el código	7
3.4. Paso 4 · El mapa: qué hay en el pom.xml	9
4. Lo que aprendiste	10
5. Para profundizar	11
6. Antes de cerrar	11

1 Antes de empezar

1.1 Qué vas a lograr

Al terminar vas a tener una aplicación Spring Boot corriendo en tu máquina, y en la consola va a salir una frase que escribiste tú. Son quince minutos y cuatro pasos, y solo se toca un archivo de código en todo el laboratorio.

Lo importante no es la frase. Es entender **quién la imprimió**. Vas a escribir un método que nadie llama desde ningún sitio, y aun así se va a ejecutar. Esa rareza —código que se ejecuta sin que tú lo llames— es la idea sobre la que está construido todo Spring, y el resto del curso consiste en aprovecharla. Hoy la vemos ocurrir por primera vez.

1.2 Qué necesitas tener listo

Solo Git. No hay que instalar Java, ni Maven, ni una base de datos: todo eso viaja dentro del repositorio del curso.

Requisito	Cómo lo compruebas	Qué tiene que salir
Git instalado	<code>git --version</code>	Cualquier versión reciente
El repositorio en tu máquina	<code>ls</code> dentro de la carpeta del curso	Se ven <code>labs/</code> , <code>docs/</code> , <code>tools/</code>
Estar en la carpeta del lab	<code>cd labs/lab-00-hola-mundo</code>	El <code>cd</code> no da error
El proyecto donde vas a trabajar	<code>ls practica</code>	Se ven <code>mvnw</code> , <code>pom.xml</code> y <code>src</code>

1.3 Cómo copiar el código de esta guía

Esto hay que decirlo antes de empezar, porque va a pasar.

Al copiar de un PDF se pierden los espacios del principio de cada línea, y a veces una línea larga se parte en dos. Es una limitación de los PDF, no de tu editor ni de esta guía.

Con Java da igual casi siempre: el compilador ignora la sangría y tu editor la vuelve a poner. Pero tiene dos consecuencias concretas que sí importan:

- **Si una línea se parte**, el editor te va a marcar un error justo ahí. Únelas y listo.
- **Con archivos .yaml la sangría es el significado.** En esta guía nunca vas a tener que pegar un bloque de YAML entero: donde toque, se te va a pedir que **cambies un valor** en el archivo que ya existe. Hazlo así aunque te tiente copiar.

Y la red de seguridad, para cualquier caso: **el código completo está en el repositorio**, en `labs/lab-00-hola-mundo/solucion/`. Si algo se pegó mal y no consigues arreglarlo, abre ahí el mismo archivo y compara.

1.4 La puesta a punto

Desde la raíz del repositorio:

```
cd labs/lab-00-hola-mundo/practica
```

Todos los comandos de esta guía se corren desde ahí. Si en algún momento algo no funciona, lo primero que hay que mirar es en qué carpeta estás.

En Windows, donde esta guía dice `./mvnw`, tú escribes `mvnw.cmd`. Es la única diferencia en todo el laboratorio.

La primera vez tarda. El repositorio trae el JDK partido en trozos, y el arranque inicial lo ensambla y comprueba su huella. Son unos segundos que solo se pagan una vez.

2 El caso

La DGT —la Dirección General de Trámites, que es el organismo ficticio que usa todo este curso— va a tener una aplicación que atienda a los contribuyentes. Hoy no atiende a nadie todavía: hoy solo **abre**.

2.1 La oficina, que es la metáfora de este laboratorio

Vas a ver esta imagen otra vez en cada paso, así que conviene presentarla bien.

Una aplicación Spring Boot es una oficina que abre por la mañana.

El `main` de tu programa es **girar la llave en la cerradura**. Es un gesto corto: una línea. Lo que pasa después no lo haces tú. Al abrir entra **el conserje** —eso es lo que Spring llama *el contenedor*—, y el conserje recorre las salas mirando qué hay preparado: qué mesas están montadas, qué carteles hay colgados, qué notas dejaron pegadas en las puertas. Lo que encuentra, lo pone en marcha.

Tú no le dices al conserje qué hacer sala por sala. **Dejas las cosas preparadas y anotadas, y él las encuentra.** Todo el curso consiste en aprender a dejar cosas preparadas de forma que el conserje sepa qué hacer con ellas.

Hoy la oficina abre, el conserje encuentra una sola nota pegada en la puerta, hace lo que dice, y la oficina cierra. Mañana —en el Lab 01— aprenderá a quedarse abierta y a atender por la ventanilla.

3 Los pasos

3.1 Paso 1 · Correr algo que todavía no es tuyo

Qué vamos a hacer

Arrancar el proyecto **sin escribir una sola línea**, para ver qué trae de fábrica y qué aspecto tiene un arranque que sale bien.

Para entenderlo mejor

La oficina ya está construida: tiene puertas, luz y llave. Todavía no has colgado nada dentro. Girar la llave y ver que abre es lo que vas a hacer ahora.

El problema

Si escribieras código primero y arrancarás después, y algo fallara, no sabrías si el fallo es tuyo o del montaje. **Arrancar antes de tocar nada te da un punto de partida que sabes que funciona**, y a partir de ahí cualquier cosa que se rompa la rompiste tú. Es la costumbre más barata que existe para no perder media hora buscando un error donde no está.

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

```

      .
     /\ /  _ _ _ _ _ ( _ ) _ _ _ _ _ \ \ \ \ \
    ( ( ) \ _ | ' _ | ' _ | ' _ \ \ _ | \ \ \ \ \
    \ \ /  _ ) | | _ | | | | | | | ( _ | | ) ) ) )
      '  | _ | . _ | | | _ | | \ _ , | / / / / /
    =====|_|=====|_|_/=/_/_/_/

:: Spring Boot ::                (v4.1.0)

... INFO ... [mi-primera-app] [main] cl.dgt.hola.HolaMundoApplication : Starting
↪ HolaMundoApplication using Java 25.0.4 ...
... INFO ... [mi-primera-app] [main] cl.dgt.hola.HolaMundoApplication : Started
↪ HolaMundoApplication in 0.495 seconds
```

Tres cosas de ahí que vale la pena mirar:

1. **El banner.** Si sale, Java compiló y Maven encontró todo lo que necesitaba. Es la señal de que el montaje está bien.
2. **using Java 25.0.4.** Ese Java viaja dentro del repositorio. No es el que tengas instalado — puede que no tengas ninguno —, y por eso en cualquier máquina dice exactamente lo mismo.
3. **Started ... in 0.495 seconds** y después el programa **termina**. No se queda esperando. Todavía no hay nada que atender.

Los tiempos y el número de PID cambian en cada corrida y en cada máquina. Si el tuyo dice 0.512 seconds, está bien. Lo que tiene que coincidir es que diga Started y no un error.

Las líneas WARNING: ... sun.misc.Unsafe ... no son un problema. Las escribe Maven, antes de que Spring exista. Aparecen en todos los laboratorios del curso y se ignoran.

Vas bien si...

Salió el banner de Spring, apareció una línea con Started HolaMundoApplication, y el programa volvió solo al prompt de la terminal sin que tuvieras que pararlo.

Si te atascas

1 · zsh: no such file or directory: ./mvnw (O command not found).

Estás en la carpeta equivocada. ./mvnw solo existe dentro de practica/ y de solucion/. Comprueba dónde estás con pwd y entra:

```
cd labs/lab-00-hola-mundo/practica
```

2 · El comando queda colgado un buen rato la primera vez.

Es normal: está ensamblando el JDK que viaja partido en el repositorio y comprobando su huella. Solo pasa la primera vez. Déjalo terminar.

3 · BUILD FAILURE y algo sobre release version 25 not supported.

Estás usando un Maven o un Java del sistema en vez del del repositorio. Asegúrate de escribir ./mvnw con el ./ delante —en Windows, mvnw.cmd—, y no mvn a secas.

3.2 Paso 2 · Que imprima algo tuyo

Qué vamos a hacer

Escribir tres líneas dentro de un método que ya está en el archivo, y ver salir tu frase en la consola.

Para entenderlo mejor

Dentro de la oficina hay una nota pegada en la puerta que dice «**lo primero al abrir, haz esto**». La nota está en blanco. Vas a escribir en ella.

Lo que importa: **el main no va a buscar esa nota**. El main solo abre. Es el conserje quien la ve al pasar, porque está pegada donde él mira, y hace lo que dice.

El problema

Un programa normal hace lo que el main le manda hacer, en el orden en que se lo manda. Si quieres que algo ocurra al arrancar, lo llamas desde el main y ya está.

Eso funciona con dos cosas. Con doscientas, el main se convierte en una lista de doscientas llamadas que hay que mantener en el orden correcto, y donde añadir algo nuevo obliga a tocar un archivo que no tiene nada que ver con lo que estás añadiendo. **El problema no es arrancar cosas: es que arrancarlas obligue a modificar el sitio donde se arranca todo lo demás.**

Cómo se hacía antes, y por qué no

Antes de Spring, esto se resolvía de una de estas dos formas, y las dos se abandonaron:

- **Lamarlo todo desde el main.** Explícito y fácil de leer con tres elementos. Con cincuenta, cualquier cambio toca el mismo archivo y todo el equipo se pelea por él.
- **Un archivo XML gigante** que declaraba qué construir y en qué orden — así se hacía en el Spring de hace quince años. Sacaba la lista del código, pero la ponía en un archivo que el compilador no revisa: escribías mal el nombre de una clase y no te enterabas hasta arrancar.

Lo que se usa hoy es **anotar la cosa donde está**. La nota va pegada en la puerta que le corresponde, no en un cuaderno central. El compilador la revisa como cualquier otro código, y añadir una no obliga a tocar nada más.

Se pega

En `practica/src/main/java/cl/dgt/hola/HolaMundoApplication.java`, **reemplazando las dos líneas de comentario** que hay dentro de `return args -> {`:

```
System.out.println();
System.out.println("  Hola, mundo. Esto lo escribí yo.");
System.out.println();
```

El método donde va, para que veas el sitio exacto — **esto no lo pegues, es para ubicarte**:

```
@Bean
CommandLineRunner run() {
    return args -> {
        System.out.println();
        System.out.println("  Hola, mundo. Esto lo escribí yo.");
        System.out.println();
    };
}
```

Ese `@Bean` de encima es la chincheta que sujeta la nota a la puerta. Y `CommandLineRunner` es el tipo de nota: «*esto se ejecuta una vez, cuando la aplicación ya está lista*».

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

```
... INFO ... [mi-primera-app] [main] cl.dgt.hola.HolaMundoApplication : Started
↪  HolaMundoApplication in 0.495 seconds

Hola, mundo. Esto lo escribí yo.
```

Fíjate en **el orden**: tu frase sale **después** de `Started`, no antes. No es casualidad ni suerte. El `CommandLineRunner` se ejecuta cuando la oficina ya está abierta y en pie — el conserje no lee la nota mientras todavía está encendiendo las luces.

Vas bien si...

Tu frase aparece en la consola, y aparece **debajo** de la línea que dice Started.

Y si te lo preguntas: en el main no hay ninguna llamada a run(). Búscala. No está. Aun así se ejecutó.

Si te atascas

1 · illegal character: '''

```
[ERROR] .../HolaMundoApplication.java:[19,32] illegal character: '''
[ERROR] .../HolaMundoApplication.java:[19,35] not a statement
[ERROR] .../HolaMundoApplication.java:[19,67] illegal character: '''
```

Es el error más común al copiar desde un PDF: se pegaron **comillas tipográficas** (“ ”) en vez de las comillas rectas de programación ("). Java no las acepta.

Borra las dos comillas de esa línea y escríbelas a mano con tu teclado. Si tu editor te las vuelve a cambiar, busca en sus preferencias «comillas inteligentes» o *smart quotes* y apágalo.

2 · <identifier> expected O illegal start of type

```
[ERROR] .../HolaMundoApplication.java:[15,23] <identifier> expected
[ERROR] .../HolaMundoApplication.java:[16,24] illegal start of type
```

Pegaste las tres líneas **fuera** del método, sueltas en el cuerpo de la clase. En Java, una instrucción tiene que vivir dentro de un método. Tienen que quedar entre return args -> { y la llave que lo cierra.

3 · ';' expected

Falta un punto y coma al final de alguna de las tres líneas, o se perdió al copiar.

4 · No sale ningún error, pero tampoco sale tu frase.

Guardaste el archivo pero no lo guardaste *de verdad*, o estás mirando la consola de una corrida anterior. Guarda, corta con Ctrl+C si hiciera falta, y vuelve a correr.

3.3 Paso 3 · Cambiar algo sin tocar el código

Qué vamos a hacer

Cambiar el nombre de la aplicación editando un archivo de configuración, sin tocar una línea de Java.

Para entenderlo mejor

El cartel de la entrada. Cambiar el nombre que se lee en la puerta no exige mover un tabique: se descuelga el cartel y se cuelga otro. **La obra y el rótulo son cosas distintas**, y conviene que lo sigan siendo.

El problema

Hay datos que cambian según dónde corra el programa: el nombre, una dirección, un número de puerto, una contraseña. Si están escritos dentro del código, cambiar cualquiera de ellos obliga

a **recompilar y volver a desplegar** — y a tener una copia distinta del programa por cada sitio donde corre. Es la forma más rápida que existe de que la versión de pruebas y la de producción dejen de ser la misma.

La alternativa, y por qué no

Podrías leerlos de variables de entorno tú mismo, con `System.getenv`, y repartir esas llamadas por el código. Funciona, pero cada sitio que lo hace inventa su propio valor por defecto, su propio nombre de variable y su propio comportamiento cuando falta.

Spring lo centraliza: un archivo, un orden de precedencia conocido, y el mismo mecanismo para todo el mundo. Lo vas a ver crecer en el Lab 13, donde el mismo programa se comporta distinto según dónde arranque **sin cambiar un byte**.

Se edita — aquí no se pega nada

Abre `practica/src/main/resources/application.yml`. Sus tres primeras líneas son éstas, y **ya están en el archivo**:

```
spring:
  application:
    name: mi-primer-app
```

Cambia únicamente el valor de `name` por lo que quieras: `la-app-de-carolina`, `la-app-de-quien-sea`. Sirve cualquier cosa sin espacios.

No pegues el bloque entero, cambia solo esa palabra. En YAML la sangría es el significado, y al copiar de un PDF la sangría se pierde. Editando el valor no corres ese riesgo, y además es lo que harías en un proyecto de verdad.

Si aun así se te desordena el archivo: cada nivel va con **dos espacios**, nunca con tabulador.

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

Lo mismo de antes, pero mira **el corchete del principio de cada línea de log**, que hasta ahora decía `[mi-primer-app]`:

```
... INFO ... [la-app-de-carolina] [main] cl.dgt.hola.HolaMundoApplication : Starting
↪ HolaMundoApplication ...
... INFO ... [la-app-de-carolina] [main] cl.dgt.hola.HolaMundoApplication : Started
↪ HolaMundoApplication in 0.4 seconds
```

Ahí está tu nombre. **No se tocó una línea de Java**.

Ojo con lo que NO cambió: sigue diciendo `Starting HolaMundoApplication`. Eso es el nombre de la **clase**. Lo del corchete es el nombre de la **aplicación**. Son dos cosas distintas y se ven juntas en la misma línea.

Vas bien si...

El corchete al principio de las líneas de log muestra el nombre que escribiste tú, y no mi-primer-app.

Si te atascas

1 · El corchete del nombre desapareció: ahora la línea empieza en [main].

```
... INFO ... [          main] cl.dgt.hola.HolaMundoApplication : Starting
↪ HolaMundoApplication ...
```

Este es el peligroso, porque no da error. Se perdió la indentación al pegar y name quedó colgando del sitio equivocado:

```
spring:
  application:
    name: la-app-de-carolina    <- mal: `name` tiene que ir MÁS adentro que
↪ `application`
```

Spring no encuentra spring.application.name, no se queja, y sigue sin nombre. name va con **cuatro** espacios, application con **dos**.

2 · found character '\t(TAB)' that cannot start any token

```
found character '\t(TAB)' that cannot start any token. (Do not use \t(TAB) for
↪ indentation)
in 'reader', line 3, column 1:
```

Hay un tabulador en el archivo. YAML no los admite en ninguna circunstancia. Bórralo y pon espacios.

3 · Cambiaste el archivo y no cambia nada.

Comprueba que editaste el de practica/ y no el de solucion/. La ruta completa es practica/src/main/resources/application.yml.

3.4 Paso 4 · El mapa: qué hay en el pom.xml

Qué vamos a hacer

Abrir el pom.xml y leer tres cosas. No se escribe nada en este paso.

Para entenderlo mejor

El pedido de material de la oficina. No dice cómo se trabaja: dice qué hay que traer para poder trabajar.

El problema

Un proyecto Java necesita bibliotecas, y las bibliotecas necesitan otras bibliotecas, y todas necesitan versiones que encajen entre sí. Elegir esas versiones a mano —y volver a elegir las cada vez que una se actualiza— es un trabajo que no aporta nada y que se hace mal muy fácilmente.

La alternativa, y por qué no

Podrías declarar cada biblioteca con su versión exacta. Es lo que había que hacer antes, y lleva a una tarde entera probando combinaciones cada vez que algo sube de versión.

Lo que hace este proyecto es heredar de un **padre**: alguien ya eligió y probó un conjunto coherente de versiones. Por eso las dependencias de abajo no llevan `<version>`.

Se lee

Abre `practica/pom.xml` y busca estas tres cosas:

En el <code>pom.xml</code>	Qué es
<code><parent> ... spring-boot-starter-parent</code>	Alguien ya eligió y probó las versiones de todo. Por eso las dependencias no llevan <code><version></code>
<code><dependency> ... spring-boot-starter</code>	Lo que el proyecto necesita para existir. Hoy hay una sola
<code><plugin> ... spring-boot-maven-plugin</code>	Lo que aporta el comando <code>spring-boot:run</code> que llevas usando

La única dependencia que tiene hoy este proyecto:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
</dependency>
```

Una línea de nada. En el Lab 01 vas a añadir una segunda, y con eso la aplicación dejará de terminar y empezará a atender por HTTP.

Vas bien si...

Puedes señalar en el `pom.xml` dónde está el padre, dónde está la única dependencia, y dónde está el plugin que da el comando que llevas usando desde el paso 1.

4 Lo que aprendiste

1 · Una aplicación Spring Boot es una clase con una anotación y un `main`.

No hay nada más. `@SpringBootApplication` sobre la clase y `SpringApplication.run(...)` dentro del `main`. Todo lo que verás en los catorce laboratorios siguientes se cuelga de esas dos líneas.

2 · Arrancarla no es correr un `main`: es levantar un contenedor.

El `main` gira la llave y se aparta. Lo que hace el trabajo es el contenedor, que recorre tu código buscando lo que está anotado y lo pone en marcha. Escribiste un método que nadie llama y se ejecutó igual — esa es la demostración, y es la idea que sostiene el framework entero.

3 · La configuración vive fuera del código.

Cambiaste el nombre de la aplicación sin tocar Java y sin recompilar nada. `application.yml` es donde van las cosas que cambian según dónde corra el programa, y ese archivo va a crecer en cada laboratorio.

4 · El `pom.xml` es la lista de lo que el proyecto necesita.

Heredar de un padre te ahorra elegir versiones. Añadir una capacidad nueva —atender HTTP, hablar con una base de datos— empieza casi siempre por añadir una línea ahí.

5 Para profundizar

Con el proyecto que ya tienes montado, y sin instalar nada:

- **Pon un segundo CommandLineRunner.** Copia el método `run()`, cámbiale el nombre al método —por ejemplo `run2()`—, deja el `@Bean`, y haz que imprima otra cosa. Arranca. Se ejecutan los dos. ¿En qué orden? ¿Puedes cambiarlo? (Pista: busca la anotación `@Order`.)
- **Quita el `@Bean` y deja el método.** Arranca. No pasa nada: el conserje solo mira lo que está anotado. Vuelve a ponerlo.
- **Cambia `logging.level.root` de `WARN` a `INFO`** en `application.yml` y arranca. Vas a ver todo lo que Spring hace al arrancar y que hoy está callado. Es mucho. Déjalo en `WARN` después.
- **Borra el `@SpringBootApplication`** y arranca. Lee el error con calma: es un buen error, y dice exactamente lo que falta.

6 Antes de cerrar

Este laboratorio **no deja nada corriendo**: la aplicación arranca, imprime y termina sola. No hay que apagar ningún proceso ni liberar ningún puerto.

Si quieres dejar la carpeta como estaba:

```
./mvnw clean
```

Eso borra `target/`, que es donde Maven deja lo compilado. No toca tu código.

Lo que te llevas, y conviene poder decirlo con tus propias palabras antes de pasar al Lab 01:

Una aplicación Spring Boot es una clase con `@SpringBootApplication` y un `main`. Al arrancar levanta un contenedor; ese contenedor encuentra lo que está anotado, y lo ejecuta.

Lo que queda pendiente, y abre el Lab 01: hoy la oficina abrió, hizo una cosa y cerró. Duró medio segundo. Una oficina que atiende a alguien tiene que **quedarse abierta y esperar** — y para esperar hay que escuchar por algún sitio: un puerto, una dirección, alguien que pregunta.

En el Lab 01 se agrega una línea al `pom.xml`, la aplicación deja de terminar, y el primer método que escribas responderá desde un navegador.